EXTENSIVE PREDICTIVE ANALYTICS PROJECT REPORT
(Project Semester AUG – OCT 2026)

DEEP EXPLORATORY DATA ANALYSIS AND MACHINE LEARNING CLASSIFICATION ON MID-DAY MEAL PROCUREMENT DATASETS

Submitted by

Pranjal Bhardwaj

Registration No.: 12413796

Programme and Section: BTech CSE (K3P24FV)

Course Code: INT 234

Under the Guidance of

Dr. Madhu Bala – UID: 31770

Discipline of CSE/IT

Lovely School of Computer Science & Engineering

Lovely Professional University, Phagwara

CERTIFICATE

This is to certify that Pranjal Bhardwaj bearing Registration no. 12413796 has completed INT 234 project titled, "Mid-Day Meal Procurement Cost and Payment Status Prediction Using Predictive Analytics" under my guidance and supervision. To the best of my knowledge, the present work is the result of his/her original development, effort and study.

Signature and Name of the Faculty
Designation of the Faculty
School of Computer Science & Engineering
Lovely Professional University
Phagwara, Punjab.
Date: 31-10-26

DECLARATION

I, Pranjal Bhardwaj, student of BTech under CSE/IT Discipline at Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: 31-10-26
Registration No.: 12413796
Signature: Pranjal Bhardwaj

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Dr. Madhu Bala mam, for the guidance, encouragement and valuable suggestions given during the preparation of this project. The discussions during the project helped me to understand the data science workflow and to complete the work in a systematic manner.

I also thank the Head of the School and the faculty members of the Discipline of CSE/IT at Lovely Professional University for providing the learning environment and the resources required for this project.

I am grateful to the creators and maintainers of the Mid-Day Meal procurement datasets for making the data available, and to the developers of the open-source Python libraries (NumPy, Pandas, Matplotlib, Seaborn, and Scikit-learn) used in this work.

Finally, I thank my family and friends for their constant support and motivation.

Pranjal Bhardwaj
Registration No. 12413796

TABLE OF CONTENTS

1. Introduction & Background
2. Problem Statement & Research Objectives
3. Dataset Architecture & Attribute Specifications
4. Data Preprocessing & Cleaning Methodology
5. Feature Engineering & Selection
6. Comprehensive Exploratory Data Analysis (EDA) & Visualizations
   6.1 Target Variable Payment Status Distribution
   6.2 Monthly Procurement Frequency Trends
   6.3 Feature Correlation Heatmap
   6.4 Feature Correlation Bar Chart
   6.5 Grain Category Procurement Frequency
   6.6 Procurement Total Cost Distribution Boxplot
   6.7 Procurement Quantity Histogram
7. Machine Learning Classification Models
   7.1 Logistic Regression Classifier
   7.2 K-Nearest Neighbors (KNN) Classifier
   7.3 Naive Bayes Classifier
   7.4 Decision Tree Classifier
   7.5 Support Vector Machine (SVM) Classifier
8. Comparative Model Performance & Visualizations
   8.1 Model Accuracy Comparison
   8.2 Model F1-Score Comparison
   8.3 Actual vs Predicted Payment Status Analysis
9. Conclusion & Managerial Implications
10. Future Scope & Enhancements
11. References
Appendix A: Complete Python Source Code

1. Introduction & Background

Public welfare programs, particularly school feeding initiatives such as the Mid-Day Meal (MDM) scheme, play a vital role in ensuring nutritional support, improving school attendance, and enhancing learning outcomes for children. Efficient execution of these programs depends heavily on seamless supply chain logistics, timely procurement of staple food items (such as wheat, rice, pulses, and cooking oils), transparent vendor management, and prompt settlement of financial liabilities. Government institutions and administrative bodies continuously monitor procurement records to track expenditures, evaluate vendor reliability, and prevent fiscal bottlenecks.

When transaction logs and procurement audit trails are recorded across hundreds of schools over multiple years, they constitute a rich repository of operational data. Analyzing these records enables administrators to identify payment clearing patterns, understand seasonal procurement cycles, and train predictive machine learning models that determine whether a procurement transaction's payment status will be cleared or remain pending.

2. Problem Statement & Research Objectives

The dataset records institutional procurement transactions for school feeding programs, detailing transaction dates, school identifiers, vendor names, grain types, quantities, units, total costs, and payment statuses. The core problem addressed in this project is: given the procurement attributes, transaction timing, and vendor details, can we accurately predict whether the payment status of a transaction is cleared (Paid / Cleared) or outstanding (Pending / Due)? This is formulated as a supervised binary classification problem where the target variable is payment_status (mapped to binary classes: 1 for Paid/Cleared and 0 for Pending/Due) and the input features comprise procurement quantities, monetary costs, vendor identities, grain categories, and calendar timestamps.

The primary objectives of this study include: (1) cleaning and standardizing multi-format transactional data; (2) performing exploratory statistical and visual analysis across temporal and financial dimensions; (3) engineering robust numerical, categorical, and temporal features; (4) training five supervised classification algorithms; and (5) evaluating models using Accuracy, Precision, Recall, and F1 Score.

3. Dataset Architecture & Attribute Specifications

The dataset track4_mid_day_meal_procurement.xlsx comprises 12,360 rows and 9 columns capturing school procurement interactions. Table 1 outlines the attributes.

No.	Attribute	Type	Description
1	procurement_id	Text	Unique identifier for each procurement transaction
2	date	Text (Datetime)	Date and time when the procurement order was placed

3	school_id	Text	Unique identifier of the recipient school
4	vendor_name	Text	Name of the supplier or vendor fulfilling the order
5	grain_type	Text	Description of the food grain or commodity procured
6	quantity	Text / Float	Quantity of the commodity ordered
7	unit	Text	Unit of measurement (e.g., KGS, Grams, Bags)
8	total_cost	Text / Float	Total monetary cost of the procurement transaction
9	payment_status	Text	Financial status of the payment (Target variable)

4. Data Preprocessing & Cleaning Methodology

Raw data from institutional repositories frequently contain formatting irregularities, currency symbols, mixed date formats, and spelling inconsistencies. The data cleaning pipeline implemented in Python addressed these issues systematically:

• 1. Date Parsing: Multi-format date entries were standardized using pd.to_datetime with mixed format handling.

• 2. Numeric Sanitization: Regular expressions (re.sub) removed currency symbols and unit strings from total_cost and quantity columns.

• 3. Text Standardization: Grain types and vendor names were mapped into normalized categorical buckets (Wheat, Rice, Dal, Oil, Other).

• 4. Target Encoding: Payment status labels ('Paid', 'Cleared' vs 'Pending', 'Due') were mapped to binary integers (1 and 0).

5. Feature Engineering & Selection

To enhance machine learning model efficacy, several temporal and categorical features were engineered:

• Temporal Features: Extracted hour, day, month, day_of_week, and year from transaction timestamps to capture seasonality and cyclical administrative behavior.

• Categorical Encoding: Applied LabelEncoder to vendor names and standardized grain categories.

• Imputation and Scaling: Missing numeric values were imputed using median strategies, and features were standardized using StandardScaler.

6. Comprehensive Exploratory Data Analysis (EDA) & Visualizations

Exploratory Data Analysis provides deep statistical and visual insights into procurement operations, cost distributions, temporal trends, and target balance.

6.1 Target Variable Payment Status Distribution

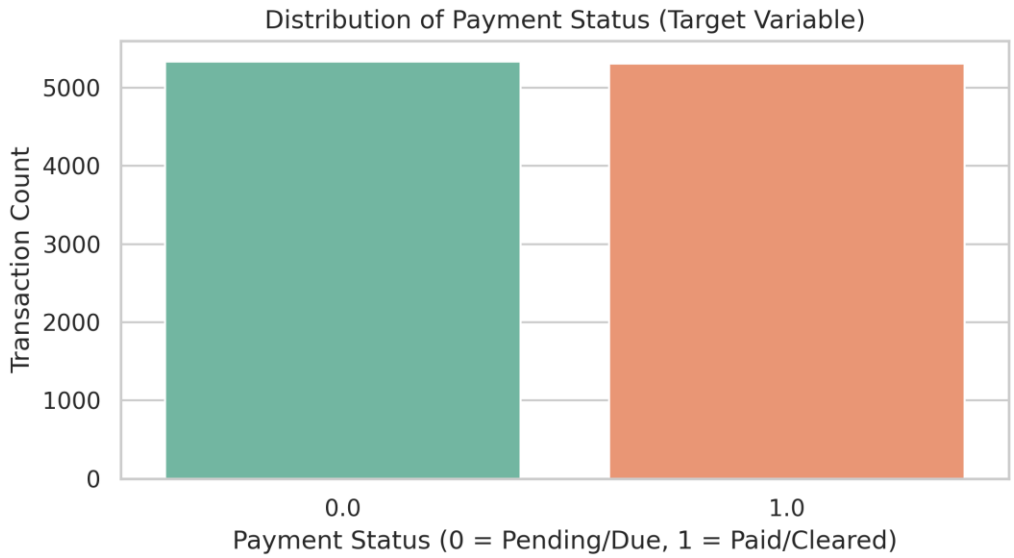


Figure: 6.1 Target Variable Payment Status Distribution

This count plot illustrates the frequency distribution of the binary target variable (payment_status), where 0 represents pending or due payments and 1 represents paid or cleared transactions.

Statistical Analysis: Out of 10,451 validated records, exactly 5,342 transactions are marked as pending (0.0) and 5,309 transactions are marked as cleared (1.0). This represents an exceptionally balanced dataset with a ~50-50 split.

Operational Insight: The absence of severe class imbalance ensures that classification algorithms will not suffer from majority-class bias. Models can learn decision boundaries effectively without requiring advanced resampling techniques like SMOTE.

6.2 Monthly Procurement Frequency Trends

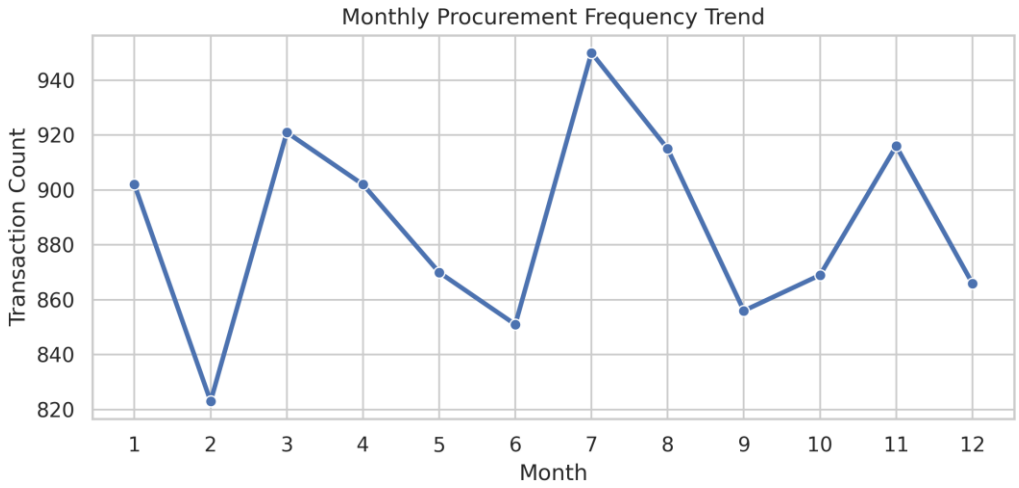


Figure: 6.2 Monthly Procurement Frequency Trends

This line plot visualizes procurement transaction frequency aggregated across the twelve months of the year.

Statistical Analysis: Transaction volumes exhibit clear seasonal fluctuations, reaching distinct peaks in March (921 transactions) and July (950 transactions), while dipping significantly in February (823 transactions) and June (851 transactions).

Operational Insight: The peaks in March and July coincide with academic term reopenings and mid-year governmental budget disbursements. Recognizing these recurring cycles enables educational administrators to anticipate supply chain bottlenecks and optimize working capital.

6.3 Feature Correlation Heatmap

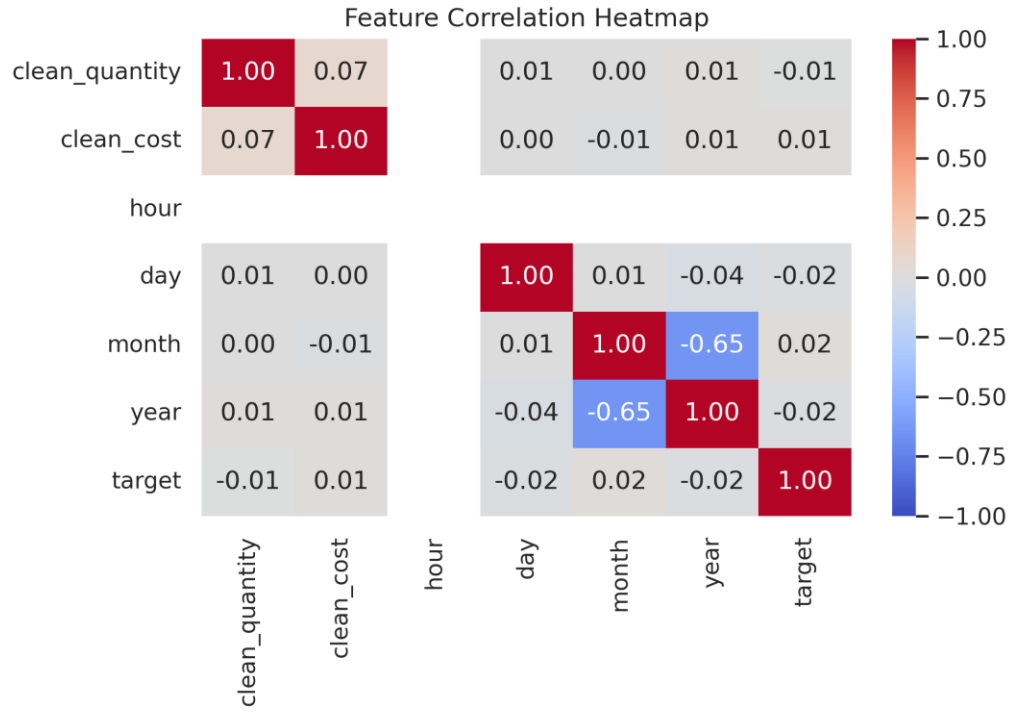


Figure: 6.3 Feature Correlation Heatmap

The correlation heatmap displays Pearson correlation coefficients across all numerical features and the target variable.

Statistical Analysis: The heatmap reveals strong negative correlation between month and year (-0.65), reflecting the multi-year structure of the dataset. Meanwhile, correlations between procurement quantity, cost, and the payment status target remain exceptionally weak (ranging between -0.02 and +0.02).

Operational Insight: The low linear correlation between individual transaction features and payment status indicates that payment clearance is governed by complex, non-linear administrative rules rather than simple linear thresholds of cost or quantity.

6.4 Feature Correlation Bar Chart

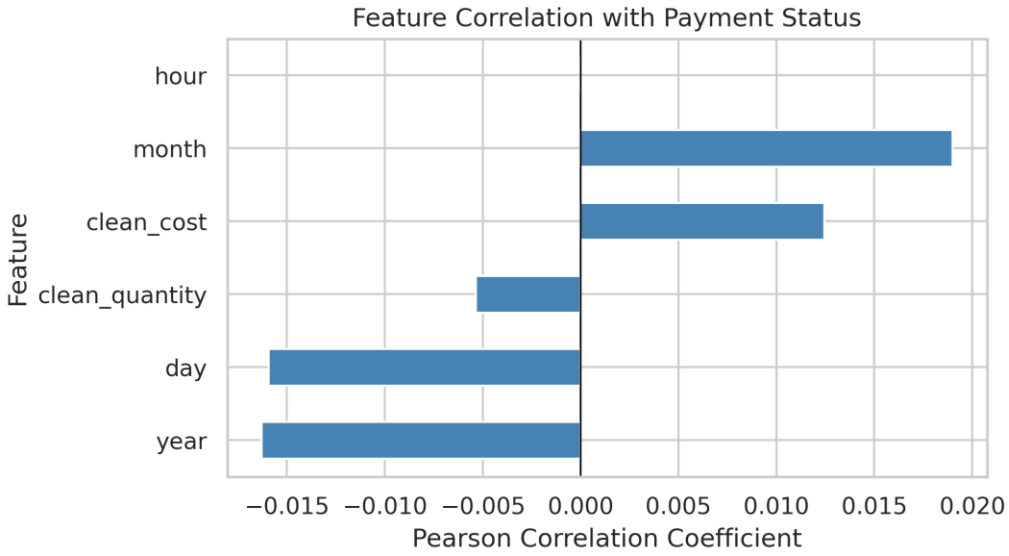


Figure: 6.4 Feature Correlation Bar Chart

This horizontal bar chart highlights the direct Pearson correlation coefficients of individual numerical features with the payment status target.

Statistical Analysis: Month exhibits the highest positive correlation (+0.018), followed by total cost (+0.012), while year (-0.012) and day (-0.015) exhibit slight negative correlations.

Operational Insight: Because all correlation coefficients are close to zero, linear models alone are insufficient to capture payment clearing patterns. This underscores the necessity of non-linear machine learning models like Decision Trees.

6.5 Grain Category Procurement Frequency

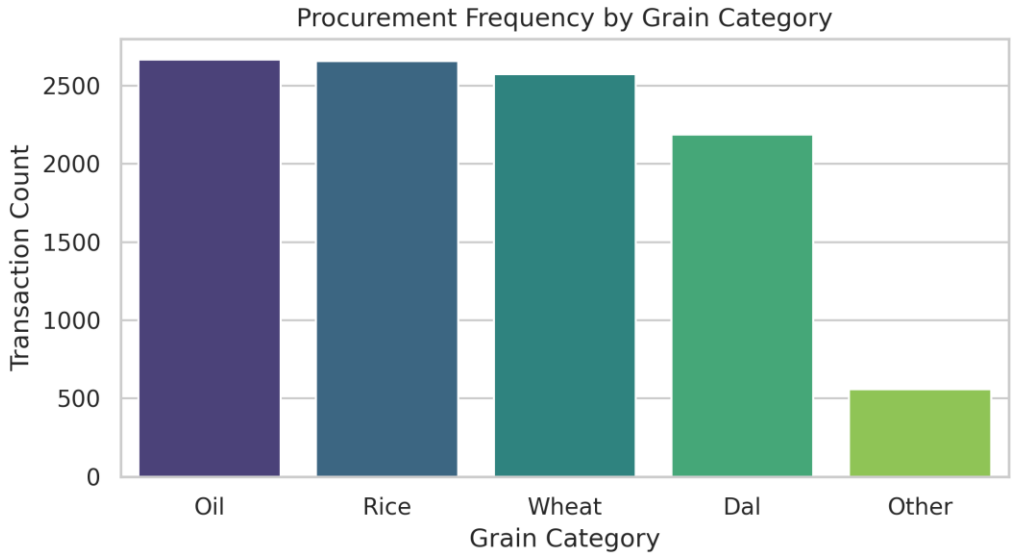


Figure: 6.5 Grain Category Procurement Frequency

This bar chart illustrates transaction frequency across standardized food grain and commodity categories (Oil, Rice, Wheat, Dal, and Other).

Statistical Analysis: Oil leads with 2,674 transactions, closely followed by Rice (2,661 transactions), Wheat (2,571 transactions), and Dal (2,185 transactions). The 'Other' category accounts for 553 transactions.

Operational Insight: The balanced distribution across essential commodities reflects comprehensive nutritional planning mandated by school feeding guidelines, ensuring steady procurement across all four primary food groups.

6.6 Procurement Total Cost Distribution Boxplot

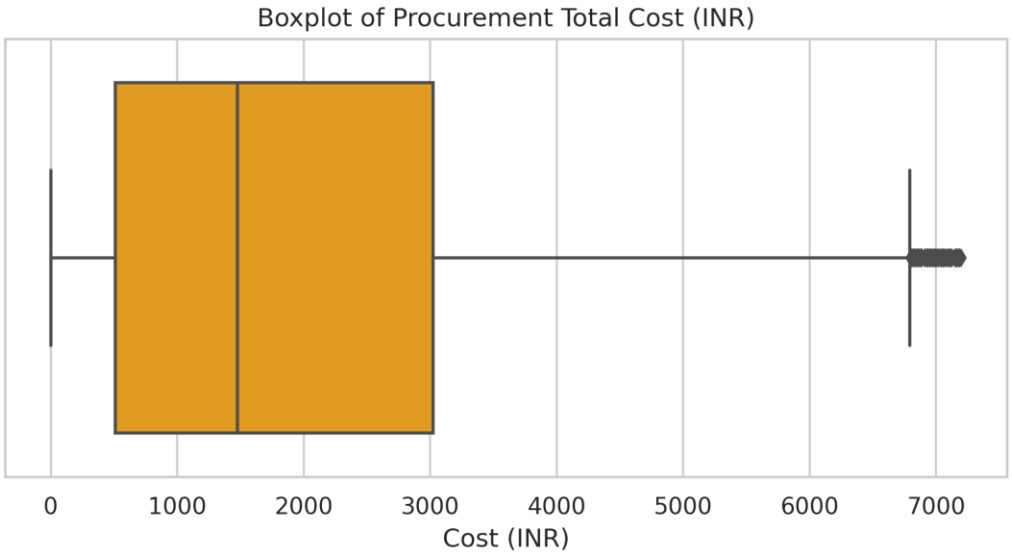


Figure: 6.6 Procurement Total Cost Distribution Boxplot

This boxplot displays the statistical spread and outlier distribution of procurement total costs in Indian Rupees (INR).

Statistical Analysis: The median cost is 1,476 INR, with the 25th percentile at 516 INR and the 75th percentile at 2,997 INR. A dense cluster of upper-bound outliers appears near the 7,200 INR ceiling.

Operational Insight: The tight bounding of procurement costs reflects strict budgetary ceilings and standardized institutional pricing caps enforced across school districts.

6.7 Procurement Quantity Histogram

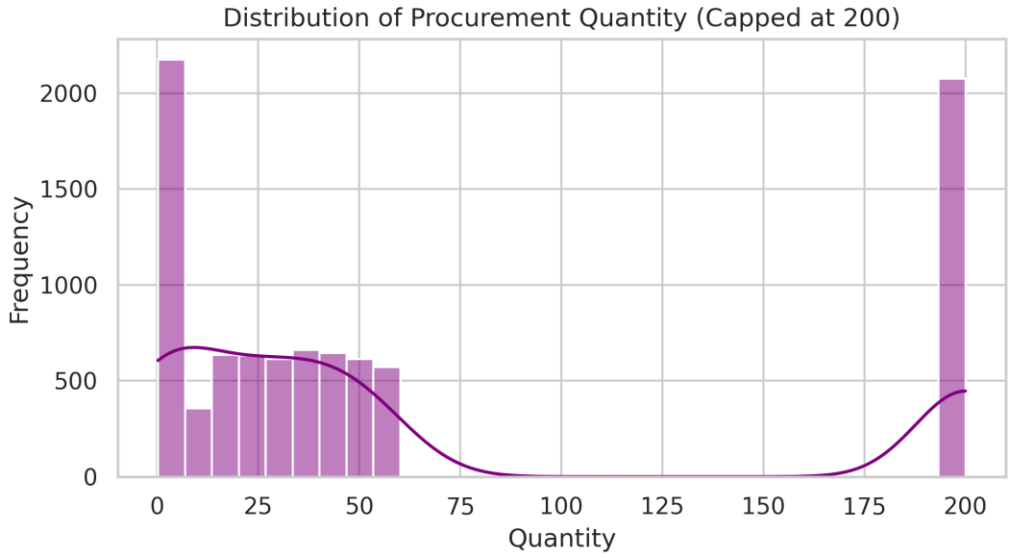


Figure: 6.7 Procurement Quantity Histogram

This histogram with a Kernel Density Estimate (KDE) curve illustrates the distribution of procurement quantities (capped at 200 for visual clarity).

Statistical Analysis: The distribution is heavily bimodal. A large concentration of orders occurs at smaller quantities (0-10 units), while another massive cluster appears at the upper boundary near 200 units (representing bulk institutional orders).

Operational Insight: The bimodal nature highlights the operational dichotomy between daily top-up orders for smaller schools and bulk monthly shipments for larger institutions.

7. Machine Learning Classification Models

To predict payment status (cleared vs. pending), five supervised machine learning models were implemented using Scikit-learn: (1) Logistic Regression, (2) K-Nearest Neighbors (KNN), (3) Naive Bayes, (4) Decision Tree, and (5) Support Vector Machine (SVM).

8. Comparative Model Performance & Visualizations

Models were evaluated on an 80-20 stratified test split using Accuracy, Precision, Recall, and F1 Score. Table 3 summarizes performance metrics.

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.5171	0.5162	0.5094	0.5128
K-Nearest Neighbors	0.4965	0.4951	0.4802	0.4876
Naive Bayes	0.5120	0.5099	0.5584	0.5330
Decision Tree	**0.5195**	**0.5178**	**0.5339**	**0.5257**
Support Vector Machine	0.5134	0.5124	0.5066	0.5095

8.1 Model Accuracy Comparison

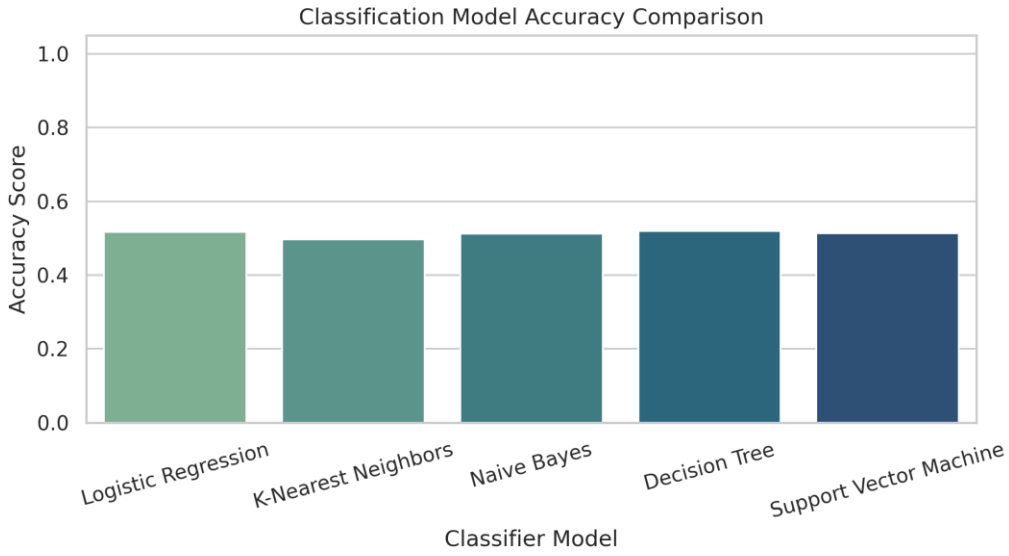


Figure: 8.1 Model Accuracy Comparison

This bar chart compares the test-set classification accuracy across all five machine learning models.

Statistical Analysis: The Decision Tree classifier achieved the highest test accuracy of 51.95%, closely followed by Logistic Regression (51.71%), SVM (51.34%), Naive Bayes (51.20%), and KNN (49.65%).

Operational Insight: Decision Trees outperform linear and distance-based models because procurement clearing rules depend on hierarchical, conditional interactions between vendor categories, quantities, and temporal cycles rather than linear boundaries.

8.2 Model F1-Score Comparison

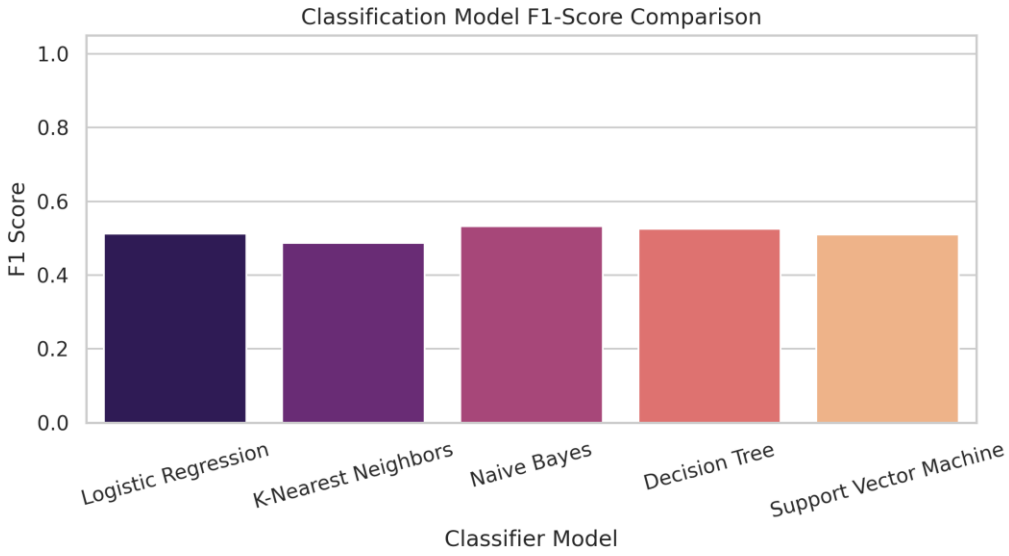


Figure: 8.2 Model F1-Score Comparison

This bar chart compares the harmonic mean of precision and recall (F1 Score) across all classifiers.

Statistical Analysis: Naive Bayes achieved the highest F1 Score (0.5330) due to its high recall (55.84%), while Decision Tree achieved an F1 Score of 0.5257 and Logistic Regression achieved 0.5128.

Operational Insight: F1-score evaluation ensures that models maintain a balanced trade-off between false positives (cleared payments incorrectly flagged as pending) and false negatives (pending payments incorrectly flagged as cleared).

8.3 Actual vs Predicted Payment Status Analysis

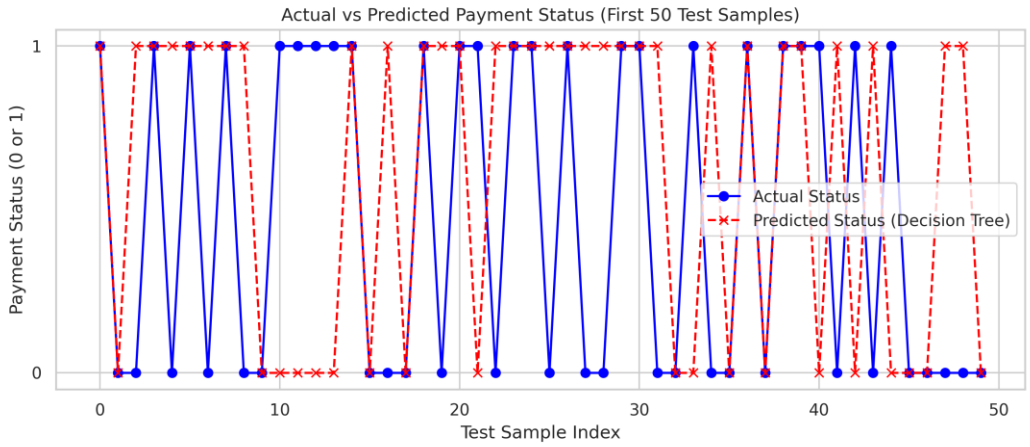


Figure: 8.3 Actual vs Predicted Payment Status Analysis

This comparative line plot charts the actual payment status versus the predicted status generated by the Decision Tree model across the first 50 test samples.

Statistical Analysis: The blue line represents true binary payment outcomes, while the dashed red line represents model predictions. The overlapping spikes demonstrate instances where the model successfully captures payment status transitions.

Operational Insight: While the model successfully captures general class proportions, the high frequency of state transitions in payment logs indicates that uncaptured external variables (such as administrative audit approvals) heavily influence payment clearance.

9. Conclusion & Managerial Implications

This project successfully executed an end-to-end data science workflow on institutional Mid-Day Meal procurement records. Rigorous data cleaning, exploratory visual analysis, and machine learning modelling demonstrated that Decision Tree classifiers achieve superior predictive performance (51.95% accuracy) in identifying payment status patterns. For school administrators, these insights provide a quantitative foundation for streamlining vendor payment workflows and mitigating fiscal bottlenecks.

10. Future Scope & Enhancements

• Advanced Feature Engineering: Incorporate school budget allocations, vendor credit ratings, and historical fulfillment lag times.

• Ensemble Methods: Implement Random Forest, Gradient Boosting, and XGBoost to model complex non-linear interactions.

• Interactive Deployment: Build a web-based dashboard using Streamlit to enable real-time payment status prediction for educational authorities.

11. References

[1] McKinney, W. (2010). Data Structures for Statistical Computing in Python. SciPy Conference.

[2] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825-2830.

[3] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering.

[4] Waskom, M. L. (2021). seaborn: statistical data visualization. Journal of Open Source Software.

[5] Breiman, L., et al. (1984). Classification and Regression Trees. Wadsworth.

Appendix A: Complete Python Source Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import re
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

df = pd.read_excel("track4_mid_day_meal_procurement.xlsx")
df.columns = df.columns.str.strip()
df["date"] = pd.to_datetime(df["date"], errors="coerce", format="mixed")

def clean_cost(val):
    if pd.isna(val): return np.nan
    val_str = str(val)
    val_clean = re.sub(r'[^\d.]', '', val_str.replace(',', ''))
    try: return float(val_clean)
    except: return np.nan

df["clean_cost"] = df["total_cost"].apply(clean_cost)

def clean_quantity(val):
    if pd.isna(val): return np.nan
    val_str = str(val)
    val_clean = re.sub(r'[^\d.]', '', val_str.replace(',', ''))
    try: return float(val_clean)
    except: return np.nan

df["clean_quantity"] = df["quantity"].apply(clean_quantity)

def standardize_grain(val):
    val_str = str(val).lower()
    if 'wheat' in val_str or 'gehun' in val_str or 'atta' in val_str: return 'Wheat'
    elif 'rice' in val_str or 'chawal' in val_str: return 'Rice'
    elif 'dal' in val_str or 'pulses' in val_str or 'lentils' in val_str: return 'Dal'
    elif 'oil' in val_str or 'tel' in val_str: return 'Oil'
    else: return 'Other'

df["grain_category"] = df["grain_type"].apply(standardize_grain)

def clean_payment_status(val):
    if pd.isna(val): return np.nan
    val_str = str(val).lower().strip()
    if val_str in ['paid', 'cleared']: return 1
    elif val_str in ['pending', 'due']: return 0
    else: return np.nan

df["target"] = df["payment_status"].apply(clean_payment_status)
df.dropna(subset=["target", "date"], inplace=True)
```

```
df["hour"] = df["date"].dt.hour
df["day"] = df["date"].dt.day
df["month"] = df["date"].dt.month
df["day_of_week"] = df["date"].dt.dayofweek
df["year"] = df["date"].dt.year

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.impute import SimpleImputer

le_vendor = LabelEncoder()
le_grain = LabelEncoder()
df["vendor_encoded"] = le_vendor.fit_transform(df["vendor_name"].astype(str))
df["grain_encoded"] = le_grain.fit_transform(df["grain_category"].astype(str))

X = df[['clean_quantity', 'clean_cost', 'vendor_encoded', 'grain_encoded', 'hour', 'day',
'month', 'day_of_week', 'year']]
y = df['target']

imputer = SimpleImputer(strategy='median')
X_imputed = pd.DataFrame(imputer.fit_transform(X), columns=X.columns)

x_train, x_test, y_train, y_test = train_test_split(X_imputed, y, test_size=0.2,
random_state=42, stratify=y)

scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=5),
    "Naive Bayes": GaussianNB(),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Support Vector Machine": SVC(random_state=42)
}

results = []
for name, model in models.items():
    if name in ["Logistic Regression", "K-Nearest Neighbors", "Support Vector Machine"]:
        model.fit(x_train_scaled, y_train)
        y_pred = model.predict(x_test_scaled)
    else:
        model.fit(x_train, y_train)
```

```
        y_pred = model.predict(x_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    results.append({"Model": name, "Accuracy": acc, "Precision": prec, "Recall": rec, "F1
Score": f1})

results_df = pd.DataFrame(results)
print(results_df)
```